Volcano Countdown

Minimax and Alpha-Beta Pruning Assignment

1. Problem Statement

A dangerous volcano is about to erupt. Two scientists, Max and Min, are competing to stop the eruption.

The volcano has a countdown value N.

- **Max plays first.**
- On each turn, a player may reduce the countdown by 1, 2, or 3.
- The countdown can never become negative.
- The player who makes the countdown exactly 0 successfully stops the eruption and wins.
- Max wants to win the game.
- Min wants to prevent Max from winning.

Your task is to determine the optimal outcome for Max using the Minimax algorithm and count how many terminal game states are evaluated when Alpha-Beta pruning is used.

2. Game Representation

The current countdown value represents the state of the game.

For example, if N = 5, Max has three possible moves:

- Remove 1 → State 4
- Remove 2 → State 3
- Remove 3 → State 2

```
MAX
/ | \
4 3 2
```

The players continue making moves until the countdown reaches 0.

3. Utility Values

- If Max makes the countdown 0, its utility is **+1**.
- If Min makes the countdown 0, its utility is **-1**.

For a terminal state:

```
state == 0 and it is Max's turn → -1
state == 0 and it is Min's turn → +1
```

The player who makes the countdown exactly 0 is the winner.

4. Minimax

- At every state where it is Max's turn, choose the maximum value among all possible moves.
- At every state where it is Min's turn, choose the minimum value among all possible moves.
- The game ends when the countdown reaches 0.

- Minimax must be implemented recursively.

Required function:

```
minimax(state, maximizing_player)
```

5. Alpha-Beta Pruning

- **alpha** = best value currently guaranteed for Max.
- **beta** = best value currently guaranteed for Min.
- A branch must be pruned whenever **alpha >= beta**.
- Moves must always be considered in the order **1** → **2** → **3**.
- Initial values are **alpha = -∞** and **beta = +∞**.

Required function:

```
alpha_beta(state, maximizing_player, alpha, beta)
```

6. Alpha-Beta State Counting

The Alpha-Beta state count is the number of terminal evaluations actually reached by the recursive Alpha-Beta algorithm.

- Increment the counter whenever a recursive call reaches **state == 0**.
- Do not count non-terminal states.
- If state 0 is reached multiple times, count each evaluation separately.
- Do not use memoization or caching to remove repeated terminal evaluations.
- The move order must remain 1 → 2 → 3.

7. Input Format

The input contains a single integer:

```
N
```

where N is the initial countdown value.

8. Constraints

```
1 ≤ N ≤ 30
```

9. Output Format

Print three values on separate lines:

```
Optimal Value: X
Best Move: Y
Alpha-Beta States: Z
```

- X is the optimal Minimax value for Max: 1 or -1.
- Y is the optimal first move for Max.
- Z is the number of terminal game states actually evaluated by Alpha-Beta.

If multiple moves give the same optimal value, choose the smallest move.

10. Sample Test Case 1

Input

```
5
```

Output

```
Optimal Value: 1
Best Move: 1
Alpha-Beta States: 10
```

Explanation: Max can remove 1 and leave 4. The position 4 is losing for the player whose turn it is, so Max can force a win. The smallest optimal move is 1.

11. Sample Test Case 2

Input

```
1
```

Output

```
Optimal Value: 1
Best Move: 1
Alpha-Beta States: 1
```

Explanation: Max removes the only remaining unit and reaches 0 immediately.

12. Sample Test Case 3

Input

```
2
```

Output

```
Optimal Value: 1
Best Move: 2
Alpha-Beta States: 2
```

Explanation: Max can remove both units immediately. Because moves are explored in the order $1 \rightarrow 2 \rightarrow 3$, the search evaluates the terminal state reached through move 1 first and then the terminal state reached through move 2.

13. Sample Test Case 4

Input

```
4
```

Output

```
Optimal Value: -1
Best Move: 1
Alpha-Beta States: 7
```

Explanation: N = 4 is a losing position for Max. Whatever Max removes, Min can respond so that Min can eventually make the countdown 0. All three first moves have value -1, so the smallest move, 1, is selected.

14. Sample Test Case 5

Input

```
6
```

Output

```
Optimal Value: 1
Best Move: 2
Alpha-Beta States: 19
```

Explanation: Max can remove 2 and leave 4 for Min. Since 4 is a losing position for the player whose turn it is, Max can force a win. Therefore the optimal first move is 2.

15. Sample Verification Summary

N	Optimal Value	Best Move	Alpha-Beta States
1	+1	1	1
2	+1	2	2
4	-1	1	7
5	+1	1	10
6	+1	2	19

16. Required Functions

1. Minimax

```
minimax(state, maximizing_player)
```

- Check whether the game has ended.
- Generate all legal moves.
- Recursively evaluate future states.
- Use max() for Max.
- Use min() for Min.

2. Alpha-Beta

```
alpha_beta(state, maximizing_player, alpha, beta)
```

- Perform the same search as Minimax.
- Maintain alpha and beta.
- Prune unnecessary branches.
- Count terminal states actually evaluated.

3. Best Move

- Determine the first move that gives Max the best possible outcome.
- If multiple moves have the same value, select the smallest move.

17. Important Requirements

- Max must always play first.

- A player may remove only 1, 2, or 3.
- Moves must be explored in the order 1 → 2 → 3.
- Minimax must be implemented recursively.
- Alpha-Beta pruning must be implemented recursively.
- Do not use any library implementing Minimax or Alpha-Beta.
- The Alpha-Beta count must include only terminal states actually evaluated.
- Repeated evaluations of state 0 are counted separately.
- Do not reorder moves to artificially increase pruning.
- The final result of Alpha-Beta must be identical to the result of Minimax.

18. Conceptual Example

For N = 4, the beginning of the game tree is:

```
MAX
/ | \
3 2 1

MIN MIN MIN
/|\ /|\ /|\
2 1 0 1 0 0 ...
```

At every level, Max selects the maximum child value while Min selects the minimum child value. Alpha-Beta pruning allows the algorithm to stop exploring a branch when it can no longer influence the final decision.

19. Complexity Requirement

- Each state can have up to 3 possible moves.
- The maximum depth is at most N.
- A loose worst-case upper bound for ordinary Minimax is O(3^N).
- Alpha-Beta pruning can significantly reduce the number of terminal states evaluated, depending on move ordering.

20. Assignment Questions

Part A — Implementation

- Implement Minimax.
- Implement Alpha-Beta pruning.
- Implement best-move selection.
- Implement the terminal-state counter.

Part B — Analysis

For each given test case, report:

- Optimal value.
- Best move.
- Number of terminal states evaluated.

- Explain the optimal strategy for Max.
- State the theoretical time complexity of Minimax.
- Explain how Alpha-Beta pruning reduces the number of terminal evaluations.